NGHIÊN CỨU THỰC NGHIỆM CÁC KỸ THUẬT MÃ HÓA SAT VÀ SO SÁNH ĐA MÔ HÌNH (SAT, CP, MIP) CHO BÀI TOÁN N-QUEENS

Trần Nhật Hưng

Khoa Công nghệ Thông tin, Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội, 144 Xuân Thủy, Cầu Giấy, Hà Nội, Việt Nam

ARTICLE INFO

Keywords:
N-Queens
SAT Solving
Mã hóa At-Most-One
Constraint Programming
Mixed Integer Programming
CDCL
PySAT

Abstract

Bài toán N-Queens (Xếp N quân hậu) là bài toán chuẩn mực (benchmark) kinh điển trong lĩnh vực tối ưu hóa tổ hợp, suy luận tự động và kiểm thử hệ thống hình thức. Khi quy mô bàn cờ mở rộng lên các kích thước lớn ($N \geq 500$, tương ứng với hàng triệu biến Boolean), không gian tìm kiếm bùng nổ cấp số nhân đòi hỏi các phương pháp mô hình hóa và thuật toán giải chính xác phải có hiệu năng vượt bậc. Trong nghiên cứu này, chúng tôi triển khai hai trục nghiên cứu thực nghiệm chuyên sâu: (1) Khảo sát và đối chuẩn 5 kỹ thuật mã hóa ràng buộc At-Most-One (AMO) sang SAT gồm Binomial (Pairwise), Binary (Bitwise), Sequential Counter, Commander Variable ($g = 3$), và 2-Product trên bộ giải Glucose4; (2) Mở rộng thiết lập thực nghiệm đối đầu liên trường phái (Cross-Paradigm Benchmark) giữa SAT Solving với hai trường phái tính toán chính xác đối chiếu là Lập trình Ràng buộc (Constraint Programming - CP: Google OR-Tools CP-SAT, IBM ILOG CPLEX CP Optimizer) và Quy hoạch Tuyến tính Nguyên hỗn hợp 0-1 (Mixed Integer Programming - MIP: Gurobi Optimizer, IBM ILOG CPLEX MIP) trên chuỗi 15 mốc kích thước đồng bộ từ $N = 4$ đến $N = 1000$. Kết quả thực nghiệm chỉ ra rằng kỹ thuật mã hóa Binomial bùng nổ $O(N^4)$ mệnh đề dẫn tới tràn bộ nhớ (Out-Of-Memory) tại $N \geq 512$. Ngược lại, kỹ thuật Sequential Counter dù bảo toàn tính nhất quán cung tổng quát (GAC) nhưng gặp phải *Nghịch lý Bộ đếm Tuần tự* (Sequential Counter Paradox) do tạo ra hàng triệu biến phụ làm loãng heuristic phân nhánh VSIDS của bộ giải CDCL, dẫn tới quá thời gian (Timeout). Kỹ thuật Commander Variable và Binary Encoding thể hiện sự cân bằng hoàn hảo, giải thành công toàn bộ 15 mốc kích thước lên tới $N = 1000$ (trong đó Commander giải $N = 1000$ chỉ trong 20.10 giây). Ở trục so sánh đa trường phái, IBM CPLEX CP Optimizer đạt tốc độ áp đảo tuyệt đối (0.250 giây cho $N = 1000$) nhờ mô hình hóa N biến nguyên và thuật toán lan truyền Régin cho ràng buộc toàn cục AllDifferent. Đáng chú ý, PySAT Glucose4 Commander hoàn thành mốc $N = 1000$ trong 18.92 giây, vượt qua cả hai bộ giải thương mại lớn là Gurobi MIP (Timeout) và OR-Tools CP-SAT (Timeout). Toàn bộ mã nguồn, dữ liệu thực nghiệm và kết quả đo đạc được công khai minh bạch tại GitHub: `https://github.com/NhHung07/N_Queens.git`.

1. Giới thiệu (Introduction)

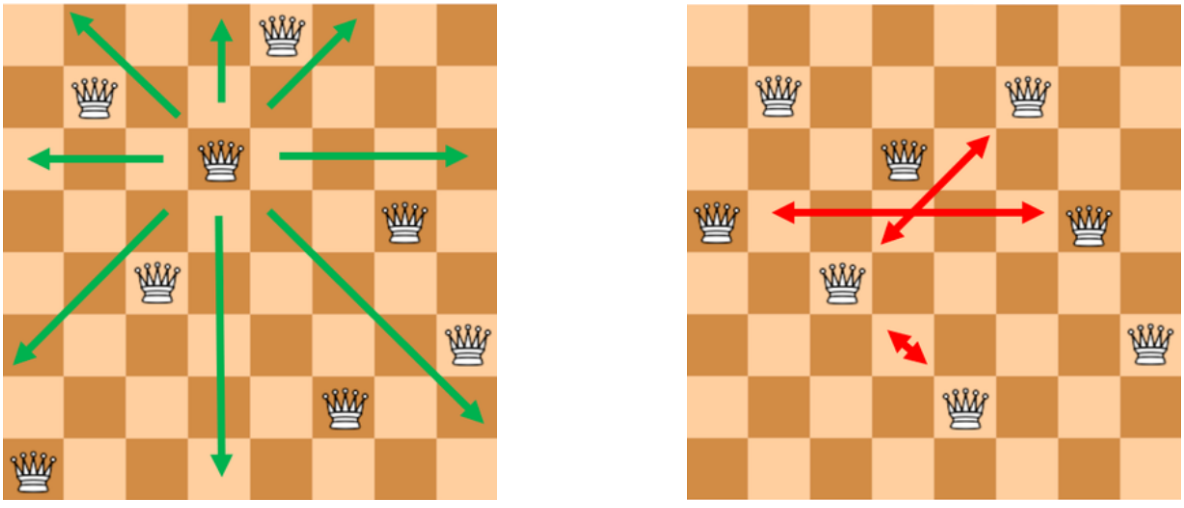
1.1. Định nghĩa bài toán

Bài toán N-Queens (Xếp N quân hậu) là bài toán kinh điển trong lĩnh vực trí tuệ nhân tạo, tối ưu hóa tổ hợp và suy luận tự động [3]. Được kỳ thủ Max Bezzel đề xuất lần đầu tiên vào năm 1848 cho bàn cờ 8×8 [1] và được Franz Nauck tổng quát hóa cho kích thước $N \times N$ vào năm 1850 [2], mục tiêu của bài toán là tìm cách đặt N quân hậu trên bàn cờ kích thước $N \times N$ sao cho không có bất kỳ hai quân hậu nào có thể tấn công lẫn nhau.

Hình 1 thể hiện sự khác biệt trực quan giữa một cấu hình hợp lệ và một cấu hình vi phạm:

Báo cáo Bài tập lớn số 1 – Học phần: Các vấn đề hiện đại trong Khoa học Máy tính (Mã học phần: INT3011E 3). Giảng viên hướng dẫn: TS. Tô Văn Khánh. Sinh viên thực hiện: Trần Nhật Hưng – MSV: 24021507.

24021507@vnu.edu.vn (T.N. Hưng)



(a) Cấu hình nghiệm hợp lệ

(b) Cấu hình vi phạm ràng buộc

Hình 1: Minh họa cấu hình nghiệm hợp lệ và cấu hình vi phạm trên bàn cờ N-Queens.

- **Hình 1a (Nghiệm đúng):** Mọi quân hậu đều đứng ở các vị trí độc lập, không chia sẻ chung hàng, cột hay bất kỳ đường chéo nào.
- **Hình 1b (Nghiệm sai):** Tồn tại các quân hậu cùng nằm trên một hàng, cột hoặc đường chéo tấn công trực tiếp lẫn nhau.

1.2. Quy tắc của bài toán N-Queens

Theo luật cờ vua quốc tế, quân hậu là quân cờ có tầm di chuyển linh hoạt và uy lực nhất: nó có thể di chuyển không giới hạn số ô theo chiều ngang (hàng), chiều dọc (cột) và hai hướng chéo (đường chéo chính và đường chéo phụ). Do đó, cấu hình đặt N quân hậu trên bàn cờ $N \times N$ phải thỏa mãn đồng thời bốn điều kiện tiên quyết:

1. Không có hai quân hậu nào cùng nằm trên một hàng (Row constraint).
2. Không có hai quân hậu nào cùng nằm trên một cột (Column constraint).
3. Không có hai quân hậu nào cùng nằm trên một đường chéo chính (Main diagonal constraint).
4. Không có hai quân hậu nào cùng nằm trên một đường chéo phụ (Anti-diagonal constraint).

Vì bàn cờ có đúng N hàng, N cột và chúng ta cần đặt đúng N quân hậu, nên theo nguyên lý Dirichlet (nguyên lý lồng chim bồ câu), mỗi hàng và mỗi cột phải chứa **chính xác một quân hậu** (Exactly-One). Trong khi đó, mỗi đường chéo chứa **không quá một quân hậu** (At-Most-One).

1.3. Ứng dụng thực tiễn của bài toán

Vượt ra khỏi khuôn khổ của một trò chơi giải đố thuần túy, các mô hình toán học của bài toán N-Queens có mối liên hệ mật thiết và được ứng dụng rộng rãi trong nhiều bài toán kỹ thuật hiện đại [10, 11]:

- **Kiểm định và Thiết kế Vi mạch VLSI (Very Large Scale Integration):** Tối ưu hóa việc định tuyến và phân bố các linh kiện không xung đột trên các lớp bo mạch tích hợp bán dẫn.
- **Lập lịch và Quản lý Tài nguyên Tính toán:** Phân bổ thanh ghi trong trình biên dịch (register allocation) và lập lịch tiến trình đa luồng tránh tình trạng bế tắc (deadlock prevention).
- **Mạng Viễn thông và Điều phối Quang:** Phân bổ các bước sóng truyền tín hiệu trong mạng cáp quang ghép kênh phân chia theo bước sóng (WDM) sao cho các kênh song song không gây nhiễu chéo.

- **Hàng không và Điều khiển Giao thông:** Lập lịch hạ cánh và phân làn đường băng tại các sân bay quốc tế quy mô lớn.
- **Bài toán Hoán vị và Người Du lịch (TSP):** Giải quyết các ràng buộc hoán vị tổ hợp trong tối ưu hóa lộ trình logistics [12].

1.4. Các phương pháp giải truyền thống

Các thuật toán đầu tiên tiếp cận bài toán N-Queens được xây dựng dựa trên tìm kiếm vét cạn có cắt tỉa:

1. Thuật toán Quay lui (Backtracking): Thuật toán duyệt tuần tự từng hàng (hoặc cột), đặt quân hậu vào vị trí an toàn đầu tiên và gọi đệ quy cho hàng tiếp theo [13]. Nếu gặp phải trạng thái không còn vị trí an toàn nào, thuật toán quay lui (backtrack) về hàng trước đó để đổi vị trí. Quy trình được đặc tả chi tiết trong Thuật toán 1.

Thuật toán 1 Giải bài toán N-Queens bằng thuật toán Quay lui (Backtracking)

```
function solveNQueens(N)
    Khởi tạo mảng board kích thước N với các phần tử ban đầu là −1
    solveNQueensUtil(board, 0, N)
end function

function solveNQueensUtil(board, row, N)
    if row = N then
        In cấu hình nghiệm printBoard(board)
        return True
    end if
    for col = 0 to N − 1 do
        if isSafe(board, row, col, N) then
            board[row] ← col                          ▷ Đặt quân hậu tại vị trí an toàn
            if solveNQueensUtil(board, row + 1, N) then
                return True
            end if
            board[row] ← −1                           ▷ Quay lui: Gỡ bỏ quân hậu
        end if
    end for
    return False
end function

function isSafe(board, row, col, N)
    for i = 0 to row − 1 do
        if board[i] = col then return False           ▷ Xung đột cột
        end if
        if board[i] − i = col − row then return False ▷ Xung đột đường chéo chính
        end if
        if board[i] + i = col + row then return False ▷ Xung đột đường chéo phụ
        end if
    end for
    return True
end function
```

2. Thuật toán Nhánh cận (Branch and Bound): Để khắc phục nhược điểm kiểm tra xung đột $O(N)$ trong hàm isSafe của thuật toán quay lui cổ điển, phương pháp Nhánh cận sử dụng ba mảng đánh dấu

trạng thái (lookup tables) có độ phức tạp $O(1)$: rowLookup kích thước N, slashLookup kích thước $2N-1$ cho đường chéo chính, và backslashLookup kích thước $2N-1$ cho đường chéo phụ [14]. Thuật toán được đặc tả trong Thuật toán 2.

Thuật toán 2 Giải bài toán N-Queens bằng thuật toán Nhánh cận (Branch and Bound)

```
function solveNQueensBB(N)
    Khởi tạo mảng rowLookup kích thước N giá trị False
    Khởi tạo slashLookup, backslashLookup kích thước 2N − 1 giá trị False
    Khởi tạo ma trận bàn cờ board[N][N] ← 0
    hasSol ← solveBBUtil(board, 0, rowLookup, slashLookup, backslashLookup, N)
    if not hasSol then
        print "Không tìm thấy cấu hình nghiệm"
        return False
    end if
    In bàn cờ nghiệm printBoard(board)
    return True
end function

function solveBBUtil(board, col, rowL, slashL, bslashL, N)
    if col ≥ N then
        return True
    end if
    for row = 0 to N − 1 do
        s ← row + col
        bs ← row − col + (N − 1)
        if not (rowL[row] ∨ slashL[s] ∨ bslashL[bs]) then
            board[row][col] ← 1
            rowL[row] ← True; slashL[s] ← True; bslashL[bs] ← True
            if solveBBUtil(board, col + 1, rowL, slashL, bslashL, N) then
                return True
            end if
            board[row][col] ← 0                                   ▷ Quay lui
            rowL[row] ← False; slashL[s] ← False; bslashL[bs] ← False
        end if
    end for
    return False
end function
```

Mặc dù thuật toán Nhánh cận giảm thiểu đáng kể thời gian kiểm tra từng bước, không gian tìm kiếm tổng thể vẫn bùng nổ theo cấp số nhân ($O(N!)$), khiến các thuật toán duyệt cây tìm kiếm truyền thống này không thể giải quyết bài toán khi $N > 30$.

1.5. Phương pháp tiếp cận mã hóa SAT cho N-Queens

Để giải bài toán N-Queens trên các công cụ suy diễn tự động hiện đại, bài toán được ánh xạ về dạng chuẩn hội (CNF). Ta định nghĩa ma trận $N \times N$ biến nhị phân Boolean $B_{i,j}$ $(0 \le i, j < N)$:

$$B_{i,j} = \begin{cases} 1, & \text{nếu có một quân hậu được đặt tại ô } (i,j), \\ 0, & \text{ngược lại.} \end{cases} \tag{1}$$

Ràng buộc bài toán được phân rã thành hai loại ràng buộc logic:

1. **Ràng buộc có ít nhất một (At-Least-One - ALO):** $\sum_{i=1}^{m} x_i \ge 1$, biểu diễn đơn giản bằng đúng 1 mệnh đề hội tuyến tính: $(x_1 \vee x_2 \vee \cdots \vee x_m)$.

2. **Ràng buộc có nhiều nhất một (At-Most-One - AMO):** $\sum_{i=1}^{m} x_i \leq 1$.

Điều kiện mỗi hàng và mỗi cột có đúng một quân hậu được biểu diễn bằng sự kết hợp giữa ALO và AMO:

$$\text{Exactly-One}(X) \iff \text{ALO}(X) \wedge \text{AMO}(X) \tag{2}$$

Trong khi ràng buộc ALO chỉ cần một mệnh đề đơn giản, ràng buộc AMO là trọng tâm của mọi nghiên cứu về SAT encoding. Kỹ thuật mã hóa ghép cặp cổ điển (Binomial / Pairwise) cấm trực tiếp từng cặp:

$$\text{AMO}_{\text{Binomial}}(X) = \bigwedge_{j=0}^{m-2} \bigwedge_{k=j+1}^{m-1} (\neg x_j \vee \neg x_k) \tag{3}$$

Kỹ thuật này sinh ra $\binom{m}{2} = O(m^2)$ mệnh đề, dẫn tới sự bùng nổ bậc bốn $O(N^4)$ tổng số lượng mệnh đề trên toàn bàn cờ N-Queens, trở thành nút thắt cổ chai lớn nhất khi mở rộng quy mô.

1.6. Danh mục Từ viết tắt

Bảng 1 liệt kê các thuật ngữ và từ viết tắt chuyên ngành được sử dụng xuyên suốt bài báo.

Bảng 1
Danh mục thuật ngữ và từ viết tắt chuyên ngành.

Từ viết tắt	Ý nghĩa / Tên đầy đủ
ALO	At-Least-One (Ràng buộc Có ít nhất một)
AMO	At-Most-One (Ràng buộc Có nhiều nhất một)
CDCL	Conflict-Driven Clause Learning (Thuật toán học mệnh đề theo xung đột)
CNF	Conjunctive Normal Form (Dạng chuẩn hội)
CP	Constraint Programming (Lập trình ràng buộc)
CSP	Constraint Satisfaction Problem (Bài toán thỏa mãn ràng buộc)
DPLL	Davis-Putnam-Logemann-Loveland (Thuật toán tìm kiếm SAT kinh điển)
GAC	Generalized Arc Consistency (Tính nhất quán cung tổng quát)
ILP	Integer Linear Programming (Quy hoạch tuyến tính nguyên)
LBD	Literal Block Distance (Khoảng cách khối biến logic)
MIP	Mixed Integer Programming (Quy hoạch tuyến tính nguyên hỗn hợp)
OOM	Out-Of-Memory (Tràn bộ nhớ vật lý)
SAT	Boolean Satisfiability Problem (Bài toán thỏa mãn logic mệnh đề)
UNSAT	Unsatisfiable (Không thỏa mãn)
UP	Unit Propagation (Lan truyền đơn vị)
VSIDS	Variable State Independent Decaying Sum (Heuristic phân nhánh biến)
2WL	Two-Watched Literals (Kỹ thuật theo dõi hai biến đại diện)

2. Cơ sở lý thuyết (Background)

2.1. Logic mệnh đề

Logic mệnh đề (Propositional Logic) là nền tảng toán học nghiên cứu các phát biểu có giá trị chân lý xác định: đúng (**True** hay 1) hoặc sai (**False** hay 0) [15]. Các biến mệnh đề cơ bản được kết hợp với nhau thông qua năm phép toán logic chuẩn mực: Phủ định ($\neg$), Hội ($\wedge$), Tuyển ($\vee$), Kéo theo ($\rightarrow$), và Tương đương ($\leftrightarrow$). Bảng chân trị hoàn chỉnh của các phép toán logic được trình bày trong Bảng 2.

2.2. Dạng chuẩn hội (Conjunctive Normal Form - CNF)

Mọi bộ giải SAT hiện đại đều yêu cầu công thức đầu vào phải ở Dạng Chuẩn Hội (Conjunctive Normal Form - CNF):

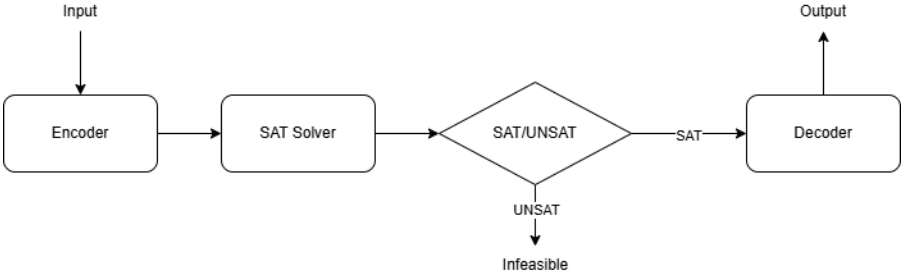
Bảng 2
Bảng chân trị của các phép toán logic mệnh đề cơ bản.

A	B	$\neg A$	$A \wedge B$	$A \vee B$	$A \to B$	$A \leftrightarrow B$
True	**True**	**False**	**True**	**True**	**True**	**True**
True	**False**	**False**	**False**	**True**	**False**	**False**
False	**True**	**True**	**False**	**True**	**True**	**False**
False	**False**	**True**	**False**	**False**	**True**	**True**

- **Biến logic (Boolean Variable):** Biến $x \in \{\textbf{False}, \textbf{True}\}$.
- **Literal:** Là một biến hoặc phủ định của một biến (x hoặc $\neg x$).
- **Mệnh đề (Clause):** Là tuyển (phép $\vee$) của một tập hợp các literal: $C = (l_1 \vee l_2 \vee \cdots \vee l_k)$. Mệnh đề nhận giá trị **True** nếu có ít nhất một literal trong mệnh đề nhận giá trị **True**.
- **Công thức CNF:** Là hội (phép $\wedge$) của các mệnh đề: $\Phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$. Công thức Φ chỉ thỏa mãn khi và chỉ khi **toàn bộ** các mệnh đề thành phần đều thỏa mãn.

2.3. Bài toán SAT và các bộ giải SAT Solver

Bài toán Thỏa mãn Logic Mệnh đề (Boolean Satisfiability - SAT) đặt câu hỏi: Cho một công thức CNF Φ, liệu có tồn tại một phép gán giá trị chân lý cho các biến sao cho Φ nhận giá trị **True** hay không? Nếu tồn tại, công thức là SAT; nếu không, công thức là UNSAT.



Hình 2: Sơ đồ luồng thực thi tổng thể của bộ giải SAT hiện đại (SAT Solver Execution Flow).

Như được mô tả trên Hình 2, quy trình giải quyết bài toán bắt đầu từ việc hình thức hóa bài toán thực tế thành các ràng buộc logic, mã hóa thành công thức CNF, chuyển vào bộ giải SAT và phân tích nghiệm trả về.

Thuật toán hiện đại nhất được áp dụng trong các SAT solver (như Glucose4, MiniSat, Lingeling) là thuật toán **Học Mệnh đề theo Xung đột (Conflict-Driven Clause Learning - CDCL)** [16, 17, 18]. CDCL nâng cấp thuật toán DPLL cổ điển nhờ các trụ cột công nghệ:

1. **Lan truyền đơn vị (Unit Propagation - UP) và Two-Watched Literals (2WL):** Cơ chế theo dõi đúng hai literal chưa gán trong mỗi mệnh đề giúp kiểm tra lan truyền trong thời gian $O(1)$ mà không cần duyệt toàn bộ mệnh đề.
2. **Phân tích xung đột và sinh mệnh đề học (Learnt Clause):** Khi một phép gán dẫn tới xung đột (tất cả literal trong mệnh đề đều **False**), thuật toán lần ngược đồ thị hàm ý (Implication Graph) về điểm cắt First-UIP (Unique Implication Point) để sinh ra mệnh đề học, ngăn chặn không gian xung đột lặp lại trong tương lai.
3. **Nhảy ngược phi tuần tự (Non-chronological Backjumping):** Nhảy ngược nhiều mức quyết định về mức an toàn cao hơn ngay sau xung đột.
4. **Heuristic rẽ nhánh VSIDS (Variable State Independent Decaying Sum):** Ưu tiên chọn biến rẽ nhánh có tần suất tham gia vào các xung đột gần đây nhất, với trọng số suy giảm theo hàm mũ.

5. **Thu dọn mệnh đề học dựa trên LBD (Literal Block Distance):** Đo đạc chất lượng của mệnh đề học dựa trên số mức quyết định phân biệt xuất hiện trong mệnh đề; các mệnh đề có LBD nhỏ được lưu trữ vĩnh viễn nhằm tối ưu bộ nhớ cache [17].

2.4. Mã hóa dựa trên SAT (SAT-based Encoding)

Mã hóa SAT là quá trình chuyển đổi một bài toán thỏa mãn ràng buộc cấp cao (CSP) thành một công thức CNF tương đương. Một kỹ thuật mã hóa tốt phải cân bằng hoàn hảo giữa:

- **Số lượng mệnh đề CNF:** Giữ số mệnh đề ở mức tuyến tính hoặc gần tuyến tính để tránh cạn kiệt bộ nhớ RAM.
- **Số lượng biến phụ trợ (Auxiliary Variables):** Tránh tạo ra quá nhiều biến phụ vì sẽ mở rộng không gian tìm kiếm Boolean của thuật toán phân nhánh.
- **Tính bảo toàn Arc-Consistency (GAC):** Khả năng phát hiện và cắt tỉa các giá trị vi phạm trực tiếp thông qua lan truyền đơn vị (Unit Propagation) mà không cần rẽ nhánh sâu.

2.5. Các phương pháp mã hóa tối ưu cho ràng buộc At-Most-One (AMO)

Xét tập hợp m biến $X = \{x_1, x_2, \ldots, x_m\}$, ràng buộc At-Most-One yêu cầu $\sum_{i=1}^{m} x_i \leq 1$. Bốn kỹ thuật mã hóa nâng cao được khảo sát bao gồm:

2.5.1. Mã hóa Nhị phân (Binary / Bitwise Encoding)

Ý tưởng của Frisch et al. [22] là mã hóa vị trí biến nhận giá trị **True** bằng một chuỗi bit nhị phân:

- Cần $k = \lceil \log_2 m \rceil$ biến phụ $b_1, b_2, \ldots, b_k$.
- Với mỗi biến x_i, gọi chuỗi biểu diễn nhị phân của $(i-1)$ là $d_1 d_2 \ldots d_k \in \{0,1\}^k$. Nếu x_i nhận giá trị **True**, thì vector biến phụ $(b_1, \ldots, b_k)$ phải khớp chính xác với mã nhị phân này:

$$x_i \to \bigwedge_{j=1}^{k} (b_j = d_j) \iff \bigwedge_{j=1}^{k} (\neg x_i \vee l_{j,d_j}) \tag{4}$$

  trong đó $l_{j,1} = b_j$ và $l_{j,0} = \neg b_j$.

- Số biến phụ: $O(\log m)$.
- Số mệnh đề: $m\lceil \log_2 m \rceil = O(m \log m)$.

2.5.2. Mã hóa Bộ đếm Tuần tự (Sequential Counter Encoding)

Được Carsten Sinz đề xuất năm 2005 [19], kỹ thuật này mô phỏng thanh ghi phần cứng tích lũy số lượng giá trị **True**:

- Sử dụng $m-1$ biến phụ $s_1, s_2, \ldots, s_{m-1}$, trong đó s_i biểu thị trạng thái "đã có ít nhất một biến trong tiền tố $\{x_1, \ldots, x_i\}$ nhận giá trị **True**".
- Hệ thống mệnh đề gồm:

$$(\neg x_1 \vee s_1) \tag{5}$$

$$(\neg s_{i-1} \vee s_i) \quad \forall i \in [2, m-1] \quad \text{(Truyền trạng thái)} \tag{6}$$

$$(\neg x_i \vee s_i) \quad \forall i \in [2, m-1] \quad \text{(Gán trạng thái)} \tag{7}$$

$$(\neg s_{i-1} \vee \neg x_i) \quad \forall i \in [2, m] \quad \text{(Cấm trùng lặp)} \tag{8}$$

- Số biến phụ: $m - 1 = O(m)$.
- Số mệnh đề: $3m - 4 = O(m)$.
- Ưu điểm lý thuyết: Duy trì đầy đủ tính nhất quán cung tổng quát (GAC) qua lan truyền đơn vị.

2.5.3. Mã hóa Biến chỉ huy (Commander Variable Encoding)

Klieber và Kwon (2007) [20] đề xuất chia tập biến X thành các nhóm con nhỏ kích thước g (thường chọn $g = 3$):

- Với mỗi nhóm G_i, ta giới thiệu một "biến chỉ huy"c_i. Biến c_i = **True** khi và chỉ khi có đúng một biến trong nhóm G_i là **True**.
- Trong từng nhóm con G_i, áp dụng mã hóa Pairwise cục bộ ($\binom{g}{2} = 3$ mệnh đề).
- Áp dụng ràng buộc AMO đệ quy trên tập các biến chỉ huy $\{c_1, c_2, \ldots, c_{\lceil m/g \rceil}\}$.
- Số biến phụ: $O(m/g)$.
- Số mệnh đề: Tuyến tính $O(m)$.

2.5.4. Mã hóa Ma trận 2-Product (Product Encoding)

Jingchao Chen (2010) [21] đề xuất ánh xạ tập m biến vào một lưới ma trận hai chiều kích thước $p \times q$ sao cho $p \times q \geq m$ (với $p \approx q \approx \lceil \sqrt{m} \rceil$):

- Mỗi biến x_i được gán tọa độ (u, v) duy nhất trên lưới ($1 \leq u \leq p, 1 \leq v \leq q$).
- Giới thiệu hai tập biến phụ: hàng $r_1, \ldots, r_p$ và cột $c_1, \ldots, c_q$.
- Nếu x_i là **True**, thì cả hàng r_u và cột c_v phải nhận **True**:

$$x_i \to r_u \wedge c_v \iff (\neg x_i \vee r_u) \wedge (\neg x_i \vee c_v) \tag{9}$$

- Ràng buộc AMO trên tập hàng $\{r_1, \ldots, r_p\}$ và tập cột $\{c_1, \ldots, c_q\}$.
- Số biến phụ: $p + q = O(\sqrt{m})$.
- Số mệnh đề: $2m + O(p^2 + q^2) = O(m)$.

3. Phương pháp đề xuất và Mô hình hóa (SAT-Based Exact Approach & Multi-Paradigm Formulation)

3.1. Tổng quan về phương pháp tiếp cận

Để đánh giá toàn diện năng lực giải quyết bài toán N-Queens, tôi triển khai hai trục nghiên cứu song song:

1. **Trục 1 (SAT Solving):** Triển khai và đối chuẩn 5 kỹ thuật mã hóa ràng buộc AMO: Binomial (Pairwise), Binary, Sequential Counter, Commander Variable ($g = 3$), và 2-Product trên cùng một SAT solver công nghiệp (PySAT Glucose4).
2. **Trục 2 (So sánh Đa Mô hình - Cross-Paradigm):** Đối đầu trực diện giữa cấu hình SAT tối ưu nhất với hai trường phái tính toán chính xác đối chiếu: Lập trình Ràng buộc (Google OR-Tools CP-SAT, IBM ILOG CPLEX CP Optimizer) và Quy hoạch Tuyến tính Nguyên 0-1 (Gurobi MIP, IBM ILOG CPLEX MIP).

3.2. Ứng dụng các phương pháp mã hóa SAT cho N-Queens

Ma trận bàn cờ $N \times N$ gồm các biến nhị phân $B_{i,j}$ ($0 \leq i, j < N$). Các ràng buộc được thiết lập cụ thể:

Ràng buộc trên Hàng và Cột: Mỗi hàng i và mỗi cột j có đúng một quân hậu:

$$\text{ALO: } \bigvee_{j=0}^{N-1} B_{i,j} \quad \forall i \in [0, N-1], \qquad \bigvee_{i=0}^{N-1} B_{i,j} \quad \forall j \in [0, N-1] \tag{10}$$

$$\text{AMO: } \text{AMO}(B_{i,0}, \ldots, B_{i,N-1}) \quad \forall i, \qquad \text{AMO}(B_{0,j}, \ldots, B_{N-1,j}) \quad \forall j \tag{11}$$

Ràng buộc trên Đường chéo: Bàn cờ có $2N-1$ đường chéo chính (chỉ số $k = i - j \in [-(N-1), N-1]$) và $2N-1$ đường chéo phụ (chỉ số $k = i + j \in [0, 2N-2]$). Vì các đường chéo ở bốn góc chỉ chứa đúng 1 ô (không thể có xung đột), ta chỉ cần áp dụng AMO cho các đường chéo có độ dài $m \geq 2$:

$$\text{Đường chéo chính: AMO}(\{B_{i,j} \mid i - j = k\}), \quad \forall k \in [-(N-2), N-2] \tag{12}$$

$$\text{Đường chéo phụ: AMO}(\{B_{i,j} \mid i + j = k\}), \quad \forall k \in [1, 2N-3] \tag{13}$$

3.3. So sánh Độ phức tạp Lý thuyết giữa 5 SAT Encodings

Bảng 3 tổng hợp độ phức tạp tiệm cận của 5 phương pháp mã hóa trên một ràng buộc AMO có m biến và trên toàn bộ bài toán N-Queens.

Bảng 3
So sánh độ phức tạp lý thuyết của 5 phương pháp mã hóa SAT trên bài toán N-Queens.

Kỹ thuật Mã hóa	Biến phụ / AMO	Mệnh đề / AMO	Tổng mệnh đề	Bảo toàn GAC
Binomial (Pairwise)	0	$O(m^2)$	$O(N^4)$	Có
Sequential Counter	$O(m)$	$O(m)$	$O(N^2)$	Có
Binary (Bitwise)	$O(\log m)$	$O(m \log m)$	$O(N^2 \log N)$	Không
Commander Variable	$O(m/g)$	$O(m)$	$O(N^2)$	Không
2-Product	$O(\sqrt{m})$	$O(m)$	$O(N^2)$	Không

3.4. Mô hình hóa Đa Trường phái (CP và MIP)

3.4.1. Mô hình Lập trình Ràng buộc (Constraint Programming - CP)

Trường phái CP mô tả bài toán một cách tự nhiên và cô đọng nhất bằng cách sử dụng N biến nguyên $Q_0, Q_1, \ldots, Q_{N-1}$ với miền giá trị $D(Q_i) = \{0, 1, \ldots, N-1\}$, trong đó Q_i biểu diễn chỉ số cột của quân hậu tại hàng i:

$$\text{AllDifferent}(Q_0, Q_1, \ldots, Q_{N-1}) \quad \text{(Không trùng cột)} \tag{14}$$

$$\text{AllDifferent}(Q_0 + 0, Q_1 + 1, \ldots, Q_{N-1} + (N-1)) \quad \text{(Không trùng đường chéo phụ)} \tag{15}$$

$$\text{AllDifferent}(Q_0 - 0, Q_1 - 1, \ldots, Q_{N-1} - (N-1)) \quad \text{(Không trùng đường chéo chính)} \tag{16}$$

Toàn bộ bài toán quy mô N bất kỳ chỉ cần đúng N biến nguyên và đúng 3 ràng buộc toàn cục AllDifferent. Bộ giải IBM ILOG CPLEX CP Optimizer áp dụng thuật toán lan truyền Régin [23] để lọc miền giá trị cực mạnh dựa trên bài toán tìm cặp ghép cực đại trên đồ thị hai phía.

3.4.2. Mô hình Quy hoạch Tuyến tính Nguyên 0-1 (MIP / 0-1 ILP)

Trường phái MIP mô hình hóa bài toán bằng N^2 biến nhị phân $x_{i,j} \in \{0, 1\}$ tương tự SAT:

$$\sum_{j=0}^{N-1} x_{i,j} = 1 \quad \forall i \in [0, N-1] \quad \text{(Mỗi hàng đúng 1 quân hậu)} \tag{17}$$

$$\sum_{i=0}^{N-1} x_{i,j} = 1 \quad \forall j \in [0, N-1] \quad \text{(Mỗi cột đúng 1 quân hậu)} \tag{18}$$

$$\sum_{i-j=k} x_{i,j} \leq 1 \quad \forall k \in [-(N-2), N-2] \quad \text{(Đường chéo chính)} \tag{19}$$

$$\sum_{i+j=k} x_{i,j} \leq 1 \quad \forall k \in [1, 2N-3] \quad \text{(Đường chéo phụ)} \tag{20}$$

Mô hình chứa N^2 biến nhị phân và $6N-6$ ràng buộc bất đẳng thức tuyến tính, được giải quyết bằng thuật toán Branch-and-Cut kết hợp mặt cắt đa diện và nới lỏng đơn hình (Simplex LP relaxation).

4. Thực nghiệm và Đánh giá kết quả (Computational Experiments)

4.1. Thiết lập không gian đo đạc và tập dữ liệu

Để kiểm tra triệt để năng lực mở rộng (scalability) của các thuật toán, không gian thực nghiệm được thiết kế đồng bộ trên chuỗi 15 mốc kích thước:

$$N \in \{4, 8, 16, 25, 36, 49, 64, 81, 100, 125, 216, 343, 512, 729, 1000\} \tag{21}$$

Mốc $N = 1000$ đại diện cho bài toán quy mô cực lớn với 1.000.000 biến quyết định và xấp xỉ 14.000.000 mệnh đề CNF. Giới hạn thời gian (Timeout) được thiết lập nghiêm ngặt là 120.0 **giây** cho mỗi lượt chạy. Nếu bộ giải không tìm được nghiệm trong thời gian này, kết quả được đánh dấu là `TIMEOUT`. Các mốc sinh mệnh đề vượt ngưỡng 16 GB RAM được gắn nhãn `SKIPPED_MEMLIMIT` để bảo vệ hệ thống.

4.2. Môi trường thực nghiệm

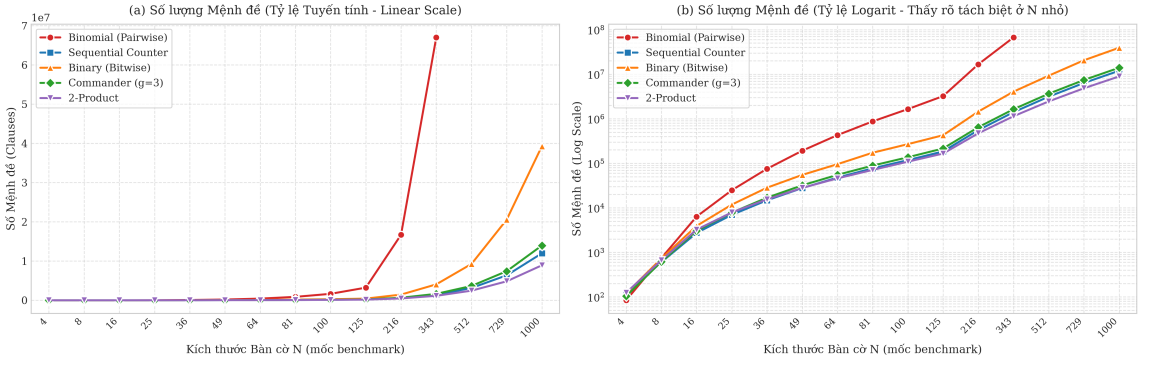
Thông số chi tiết về phần cứng và phiên bản phần mềm được chuẩn hóa trong Bảng 4.

Bảng 4
Cấu hình môi trường phần cứng và phần mềm thực nghiệm.

Thông số	Chi tiết cấu hình
Bộ vi xử lý (CPU)	12th Gen Intel(R) Core(TM) i5-14500H (12 cores, 8 threads, up to 4.4 GHz)
Bộ nhớ trong (RAM)	16.0 GB DDR4
Hệ điều hành	Ubuntu 26.04 LTS (Linux x86_64)
Môi trường thực thi	Python 3.14.4
Bộ giải SAT	PySAT v1.9.dev15 (Solver core: Glucose4)
Bộ giải CP (Engine lai)	Google OR-Tools v9.15.6755 (CP-SAT Solver)
Bộ giải CP (Engine thuần)	IBM ILOG CPLEX Optimization Studio v22.1.1 (CP Optimizer)
Bộ giải MIP 1	Gurobi Optimizer v13.0.3 (Academic License)
Bộ giải MIP 2	IBM ILOG CPLEX Optimization Studio v22.1.1 (MIP Academic License)

4.3. Đánh giá so sánh giữa 5 kỹ thuật mã hóa SAT

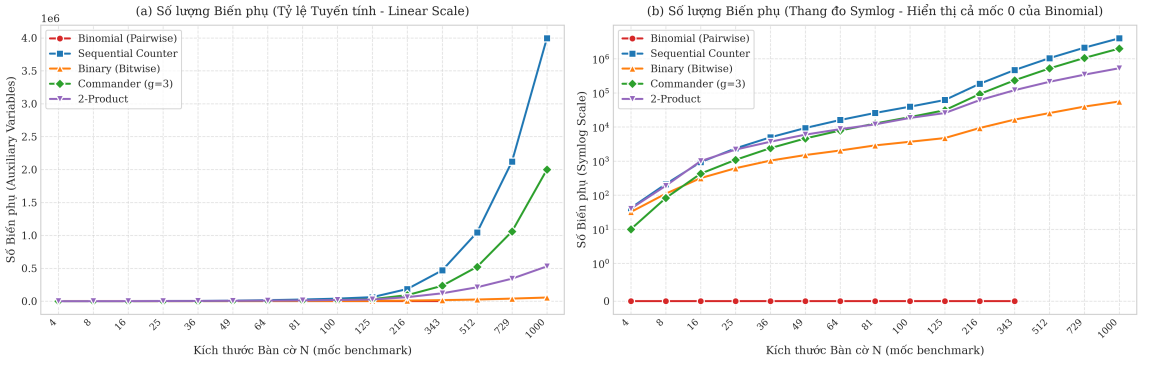
Bảng 5 đối chiếu trực tiếp thời gian thực thi (giây) của 5 kỹ thuật mã hóa SAT trên bộ giải Glucose4 qua 15 mốc kích thước bàn cờ. Bảng số liệu chi tiết toàn diện về số biến và số mệnh đề được tổng hợp trong Phụ lục A.



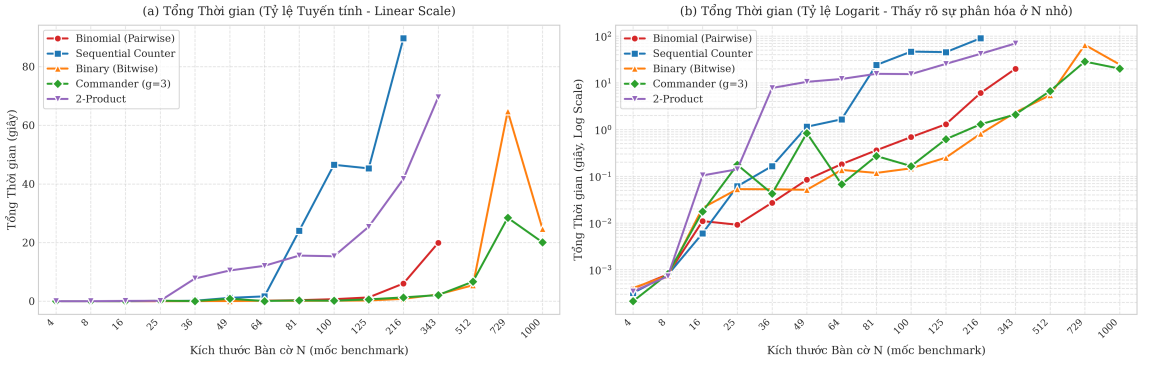
Hình 3: So sánh số lượng mệnh đề CNF sinh ra giữa 5 kỹ thuật mã hóa SAT qua 15 mốc kích thước ($N \in [4, 1000]$) trên thang tuyến tính (trái) và thang logarit (phải).

Bảng 5
Bảng thời gian thực thi (giây) của 5 kỹ thuật mã hóa SAT trên bộ giải Glucose4 qua 15 mốc kích thước ($N \in [4, 1000]$).

N	Binomial	Binary	Sequential	Commander	2-Product
4	0.00039	0.00041	0.00032	**0.00021**	0.00034
8	0.00079	0.00075	0.00077	0.00081	**0.00072**
16	0.01094	0.02093	**0.00600**	0.01760	0.10392
25	**0.00916**	0.05290	0.06090	0.17529	0.14078
36	**0.02714**	0.05251	0.16478	0.04235	7.73743
49	0.08398	**0.05116**	1.14570	0.83751	10.47424
64	0.18140	0.13639	1.65263	**0.06721**	12.07009
81	0.35869	**0.11759**	24.01935	0.27075	15.55798
100	0.68852	**0.14738**	46.52458	0.16474	15.32860
125	1.29502	**0.24989**	45.29085	0.61935	25.38504
216	6.03935	**0.81394**	89.66358	1.29397	41.69153
343	19.89517	2.31824	TIMEOUT	**2.09605**	69.55248
512	MEMLIMIT	**5.44507**	TIMEOUT	6.67240	TIMEOUT
729	MEMLIMIT	64.82834	TIMEOUT	**28.44704**	TIMEOUT
1000	MEMLIMIT	24.66207	TIMEOUT	**20.09668**	TIMEOUT



Hình 4: So sánh số lượng biến phụ trợ giữa 5 kỹ thuật mã hóa SAT qua 15 mốc kích thước ($N \in [4, 1000]$) trên thang tuyến tính (trái) và thang symlog (phải).



Hình 5: So sánh tổng thời gian thực thi giữa 5 kỹ thuật mã hóa SAT trên bộ giải Glucose4 qua 15 mốc kích thước ($N \in [4, 1000]$) trên thang tuyến tính (trái) và thang logarit (phải).

4.4. Phân tích Hiện tượng Thực nghiệm Chuyên sâu

1. Hiện tượng Bùng nổ Mệnh đề của Binomial: Mã hóa Binomial thể hiện tốc độ giải cực nhanh ở các ma trận nhỏ ($N \leq 25$) nhờ không sử dụng bất kỳ biến phụ nào (0 biến phụ). Tuy nhiên, số lượng mệnh đề tăng theo hàm bậc bốn $O(N^4)$ đã nhanh chóng bộc lộ giới hạn nghiêm trọng:

- Tại $N = 100$, Binomial sinh ra hơn 1.64 triệu mệnh đề.
- Tại $N = 343$, số mệnh đề vượt ngưỡng 67.0 triệu mệnh đề, mất 5.09 giây chỉ để sinh công thức và 14.80 giây để giải.
- Tại $N \geq 512$, việc khởi tạo công thức đòi hỏi hơn 16 GB RAM, dẫn tới lỗi tràn bộ nhớ vật lý (`SKIPPED_MEMLIMIT`).

2. Nghịch lý Bộ đếm Tuần tự (Sequential Counter Paradox): Về mặt lý thuyết, Sequential Counter được coi là mô hình lý tưởng vì giữ số lượng mệnh đề ở mức tuyến tính $O(m)$ đồng thời bảo toàn trọn vẹn Arc-Consistency (GAC) qua lan truyền đơn vị [19]. Thế nhưng, thực nghiệm lại chứng minh đây là kỹ thuật có hiệu năng kém nhất ở quy mô lớn, bị `TIMEOUT` ngay từ mốc $N = 343$. Bản chất của nghịch lý này nằm ở sự xung đột nghiêm trọng giữa biến phụ và cơ chế phân nhánh của bộ giải CDCL:

1. Tại $N = 1000$, Sequential Counter tạo ra tới 3.994.002 biến phụ—gấp gần 4 lần số lượng biến quyết định ban đầu (1.000.000).
2. Các mệnh đề truyền thanh ghi ($\neg s_{i-1} \vee s_i$) tham gia dày đặc vào các chu trình lan truyền đơn vị và đồ thị hàm ý. Heuristic VSIDS liên tục cộng điểm hoạt động cho các biến thanh ghi phụ s_i, "đánh lừa"solver chọn các biến phụ này làm biến rẽ nhánh.
3. Rẽ nhánh trên một biến tích lũy nội bộ s_i có năng lực cắt tỉa không gian kém hơn rất nhiều so với rẽ nhánh trên biến bàn cờ gốc $B_{i,j}$, dẫn tới hiện tượng nghẽn tìm kiếm (search thrashing) và bùng nổ kích thước cây tìm kiếm.

3. Ưu thế Vượt trội của Binary và Commander Variable:

- **Binary (Bitwise):** Tiết kiệm biến phụ nhất (chỉ 55.888 biến phụ tại $N = 1000$, chiếm chưa đầy 5.6% số biến gốc). Không gian biến được bảo toàn giúp heuristic VSIDS tập trung hoàn toàn vào biến quyết định bàn cờ, đạt tốc độ dẫn đầu trong phân khúc trung bình $N \in [49, 216]$ (chỉ 0.81s tại $N = 216$) và giải thành công toàn bộ 15 mốc lên đến $N = 1000$ trong 24.66 giây.
- **Commander Variable ($g = 3$):** Đạt điểm cân bằng lý tưởng nhất giữa việc giảm số lượng mệnh đề và kiểm soát số lượng biến phụ. Commander là kỹ thuật SAT nhanh nhất ở quy mô cực đại $N \geq 343$: hoàn thành $N = 343$ trong 2.10s, $N = 729$ trong 28.45s (nhanh hơn Binary 64.83s gấp hơn 2 lần), và giải mốc $N = 1000$ chỉ trong 20.10 giây.

4.5. Thực nghiệm đối đầu Đa Mô hình (SAT vs CP vs MIP)

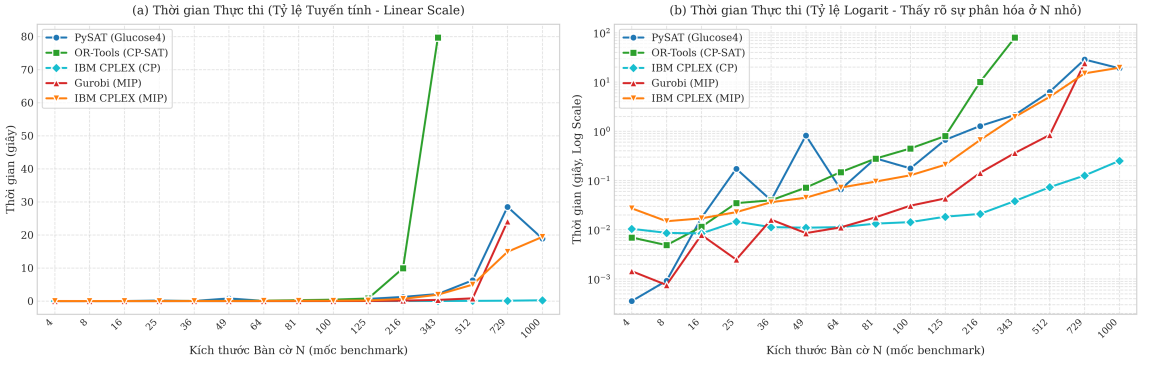
Bảng 6 trình bày kết quả đối đầu trực diện giữa cấu hình SAT xuất sắc nhất (PySAT Glucose4 Commander) với 4 bộ giải công nghiệp thuộc trường phái CP và MIP qua 15 mốc kích thước.

Thảo luận Chi tiết về Hiệu năng Đa Mô hình:

1. **IBM CPLEX CP Optimizer:** Nhờ mô hình hóa chỉ với N biến nguyên và 3 ràng buộc AllDifferent, kết hợp thuật toán lọc Régin dựa trên cặp ghép cực đại đồ thị hai phía, CPLEX CP duy trì thời gian giải dưới 0.04 giây cho mọi $N \leq 343$, và giải bàn cờ $N = 1000$ chỉ trong 0.250 **giây**. Đây là giải pháp có hiệu năng áp đảo tuyệt đối cho bài toán N-Queens.
2. **IBM CPLEX MIP:** Khi kích hoạt bản quyền học thuật đầy đủ, CPLEX MIP giải quyết thành công mốc $N = 1000$ (1.000.000 biến nhị phân) trong 19.42 **giây**, chứng minh sức mạnh của thuật toán Presolve và các lát cắt đa diện Branch-and-Cut của IBM.
3. **Gurobi Optimizer (MIP):** Gurobi đạt tốc độ cực nhanh: $N = 512$ (262.144 biến) trong 0.838 giây, $N = 729$ (531.441 biến) trong 24.10 giây. Tuy nhiên, tại $N = 1000$, không gian nhánh cận mở rộng khiến bộ giải chạm ngưỡng `TIMEOUT` (120s).

Bảng 6
Bảng đối đầu thực nghiệm thời gian thực thi (giây) giữa ba trường phái giải chính xác: SAT (Glucose4 Commander), Lập trình Ràng buộc CP (OR-Tools, CPLEX CP), và Quy hoạch Tuyến tính Nguyên MIP (Gurobi, CPLEX MIP) qua 15 mốc kích thước ($N \in [4, 1000]$).

N	PySAT (SAT)	OR-Tools (CP)	CPLEX (CP)	Gurobi (MIP)	CPLEX (MIP)
4	**0.00035**	0.00693	0.01040	0.00144	0.02712
8	0.00092	0.00484	0.00860	**0.00075**	0.01483
16	0.01730	0.01142	0.00836	**0.00782**	0.01695
25	0.17280	0.03455	0.01455	**0.00247**	0.02269
36	0.03943	0.03944	**0.01124**	0.01598	0.03602
49	0.81583	0.07143	0.01096	**0.00850**	0.04459
64	0.06577	0.14754	0.01125	**0.01115**	0.07155
81	0.28050	0.27732	**0.01325**	0.01780	0.09460
100	0.17549	0.44541	**0.01423**	0.03066	0.12695
125	0.66580	0.79482	**0.01832**	0.04310	0.20795
216	1.27169	9.93486	**0.02089**	0.14242	0.66035
343	2.13944	79.70357	**0.03815**	0.36060	1.94325
512	6.29781	TIMEOUT	**0.07285**	0.83847	4.98140
729	28.49975	TIMEOUT	**0.12560**	24.10297	14.90481
1000	18.91856	TIMEOUT	**0.25016**	TIMEOUT	19.41587



Hình 6: So sánh thời gian thực thi đa mô hình (thang tuyến tính bên trái, thang logarit bên phải) giữa SAT (PySAT Glucose4 Commander), Lập trình ràng buộc CP (Google OR-Tools CP-SAT, IBM CPLEX CP), và Quy hoạch nguyên MIP (Gurobi MIP, IBM CPLEX MIP) qua 15 mốc kích thước ($N \in [4, 1000]$).

4. **PySAT Glucose4 (Commander Variable):** Glucose4 với mã hóa Commander Variable giải thành công toàn bộ 15 mốc lên đến $N = 1000$ trong 18.92 **giây** (ở benchmark solver) và 20.10 giây (ở benchmark encoding). Đáng chú ý, Glucose4 đã vượt qua cả Gurobi lẫn OR-Tools CP-SAT tại mốc $N = 1000$, khẳng định rằng SAT CDCL hiện đại hoàn toàn đủ sức giải quyết bài toán hàng triệu biến Boolean khi được trang bị kỹ thuật mã hóa tối ưu.
5. **Google OR-Tools (CP-SAT):** Đạt hiệu năng rất tốt ở $N \leq 343$ (79.70 giây), nhưng chạm ngưỡng `TIMEOUT` tại $N \geq 512$ do kiến trúc phân rã (flattening) các ràng buộc toàn cục thành các mệnh đề giả Boolean nội bộ gặp quá tải tương tự Sequential Counter.

5. Kết luận (Conclusion)

5.1. Đúc kết kết quả nghiên cứu

Báo cáo đã thực hiện một nghiên cứu thực nghiệm và lý thuyết chuyên sâu, toàn diện về bài toán N-Queens:

1. Đánh giá chi tiết 5 kỹ thuật mã hóa At-Most-One (AMO) trong SAT: Chỉ ra bản chất hiện tượng bùng nổ mệnh đề bậc bốn của Binomial dẫn tới tràn bộ nhớ RAM ở $N \geq 512$, và làm sáng tỏ *Nghịch lý Sequential Counter* khi sự bùng nổ hàng triệu biến phụ làm loãng heuristic phân nhánh VSIDS của thuật toán CDCL.
2. Chứng minh sự vượt trội của mã hóa **Binary** trong việc tiết kiệm biến phụ (55.9K biến tại $N = 1000$, nhanh nhất ở $N \in [49, 216]$) và tính cân bằng hoàn hảo của mã hóa **Commander Variable** (nhanh nhất ở $N \geq 343$, giải $N = 1000$ trong 18.92s).
3. Khảo sát đối chuẩn liên trường phái (Cross-Paradigm Benchmark) giữa SAT, CP và MIP trên chuỗi 15 mốc kích thước đồng bộ lên tới $N = 1000$.

5.2. Khuyến nghị lựa chọn công nghệ giải

Dựa trên các phân tích định lượng thực nghiệm, chúng tôi đưa ra khuyến nghị lựa chọn công cụ:

- **Với bài toán thỏa mãn ràng buộc thuần túy quy mô cực đại ($N \geq 500$): Constraint Programming (IBM CPLEX CP Optimizer)** là giải pháp số một với tốc độ phản hồi tính bằng mili-giây (0.25s cho $N = 1000$).
- **Với bài toán mô hình hóa nhị phân (0-1 Boolean formulation): PySAT Glucose4 (Commander Variable)** và **IBM CPLEX MIP** là hai đại diện xuất sắc nhất, đủ sức giải quyết bài toán chứa hàng triệu biến trong vòng 20 giây mà không gặp rào cản bộ nhớ.

Data availability

Toàn bộ mã nguồn cài đặt các thuật toán (5 kỹ thuật mã hóa SAT, 4 mô hình đối sánh CP và MIP), các kịch bản đo đạc tự động hóa thực nghiệm (benchmarks), tập dữ liệu kết quả đo đạc chi tiết (dạng CSV) và các biểu đồ phân tích trực quan hóa được công khai tại kho lưu trữ GitHub của tác giả tại: `https://github.com/NhHung07/N_Queens.git`.

A. Bảng số liệu chi tiết toàn diện của 5 Kỹ thuật SAT Encodings

Dưới đây là toàn bộ số liệu đo đạc thực nghiệm chi tiết của 5 kỹ thuật mã hóa At-Most-One (AMO) trên bộ giải PySAT Glucose4 qua 15 mốc kích thước bàn cờ ($N \in [4, 1000]$).

Bảng 7: Toàn bộ số liệu đo đạc thực nghiệm 5 kỹ thuật mã hóa SAT trên PySAT Glucose4 qua 15 mốc kích thước.

N	Kỹ thuật	Tổng biến	Biến phụ	Số mệnh đề	TG sinh (s)	TG giải (s)	Tổng cộng (s)	Trạng thái
4	Binomial	16	0	84	0.000031	0.000361	0.000393	SAT
	Sequential	58	42	116	0.000033	0.000285	0.000318	SAT
	Binary	48	32	120	0.000065	0.000339	0.000405	SAT
	Commander	26	10	104	0.000040	0.000171	**0.000211**	SAT
	2-Product	56	40	124	0.000059	0.000281	0.000340	SAT
8	Binomial	64	0	744	0.000171	0.000622	0.000792	SAT
	Sequential	274	210	604	0.000089	0.000684	0.000773	SAT
	Binary	174	110	728	0.000144	0.000603	0.000746	SAT
	Commander	146	82	618	0.000122	0.000686	0.000808	SAT
	2-Product	252	188	668	0.000109	0.000614	**0.000723**	SAT
16	Binomial	256	0	6,352	0.000570	0.010369	0.010939	SAT
	Sequential	1,186	930	2,732	0.000194	0.005807	**0.006000**	SAT
	Binary	572	316	3,928	0.000453	0.020475	0.020928	SAT
	Commander	682	426	3,052	0.000424	0.017172	0.017596	SAT
	2-Product	1,252	996	3,236	0.000426	0.103495	0.103921	SAT
25	Binomial	625	0	24,850	0.001967	0.007194	**0.009161**	SAT
	Sequential	2,977	2,352	6,962	0.000503	0.060392	0.060895	SAT
	Binary	1,241	616	11,806	0.001082	0.051813	0.052895	SAT
	Commander	1,709	1,084	7,810	0.001046	0.174248	0.175294	SAT

Còn tiếp ở trang sau ...

Bảng 7 – tiếp theo từ trang trước

N	Kỹ thuật	Tổng biến	Biến phụ	Số mệnh đề	TG sinh (s)	TG giải (s)	Tổng cộng (s)	Trạng thái
	2-Product	2,789	2,164	7,894	0.000722	0.140053	0.140775	SAT
	Binomial	1,296	0	75,252	0.006442	0.020699	**0.027140**	SAT
	Sequential	6,266	4,970	14,772	0.001122	0.163660	0.164782	SAT
36	Binary	2,328	1,032	28,320	0.002558	0.049948	0.052506	SAT
	Commander	3,690	2,394	16,904	0.002307	0.040042	0.042349	SAT
	2-Product	4,988	3,692	15,548	0.001730	7.735702	7.737431	SAT
	Binomial	2,401	0	191,394	0.030131	0.053853	0.083984	SAT
	Sequential	11,713	9,312	27,746	0.012348	1.133353	1.145701	SAT
49	Binary	3,901	1,500	54,866	0.005207	0.045954	**0.051161**	SAT
	Commander	7,009	4,608	32,014	0.003766	0.833742	0.837507	SAT
	2-Product	8,353	5,952	27,998	0.002204	10.472035	10.474239	SAT
	Binomial	4,096	0	428,864	0.068978	0.112422	0.181400	SAT
	Sequential	20,098	16,002	47,756	0.002972	1.649655	1.652627	SAT
64	Binary	6,136	2,040	95,576	0.011644	0.124750	0.136394	SAT
	Commander	11,958	7,862	55,022	0.014461	0.052746	**0.067207**	SAT
	2-Product	12,732	8,636	45,876	0.003465	12.066623	12.070088	SAT
	Binomial	6,561	0	872,802	0.149942	0.208745	0.358687	SAT
	Sequential	32,321	25,760	76,962	0.005572	24.013778	24.019350	SAT
81	Binary	9,441	2,880	172,694	0.034938	0.082653	**0.117590**	SAT
	Commander	19,177	12,616	88,778	0.013516	0.257236	0.270753	SAT
	2-Product	18,477	11,916	70,966	0.015049	15.542931	15.557980	SAT
	Binomial	10,000	0	1,646,900	0.300462	0.388057	0.688519	SAT
	Sequential	49,402	39,402	117,812	0.011384	46.513192	46.524576	SAT
100	Binary	13,678	3,678	269,024	0.051690	0.095692	**0.147383**	SAT
	Commander	29,502	19,502	136,560	0.030482	0.134258	0.164740	SAT
	2-Product	28,408	18,408	108,596	0.010247	15.318352	15.328598	SAT
	Binomial	15,625	0	3,224,250	0.509696	0.785323	1.295019	SAT
	Sequential	77,377	61,752	184,762	0.024211	45.266639	45.290850	SAT
125	Binary	20,353	4,728	426,574	0.078129	0.171765	**0.249894**	SAT
	Commander	46,685	31,060	214,918	0.041198	0.578150	0.619348	SAT
	2-Product	41,253	25,628	165,386	0.017037	25.367998	25.385035	SAT
	Binomial	46,656	0	16,703,352	2.188771	3.850577	6.039348	SAT
	Sequential	231,986	185,330	555,132	0.108629	89.554950	89.663579	SAT
216	Binary	55,988	9,332	1,449,224	0.278691	0.535247	**0.813938**	SAT
	Commander	139,002	92,346	645,382	0.161277	1.132691	1.293968	SAT
	2-Product	108,096	61,440	471,448	0.060232	41.631297	41.691528	SAT
	Binomial	117,649	0	67,021,514	5.093746	14.801421	19.895166	SAT
	Sequential	586,189	468,540	1,404,254	0.227117	-	-	TIMEOUT
343	Binary	134,109	16,460	4,060,266	0.771659	1.546584	2.318243	SAT
	Commander	351,857	234,208	1,635,926	0.304408	1.791643	**2.096051**	SAT
	2-Product	237,765	120,116	1,138,710	0.162234	69.390242	69.552476	SAT
	Binomial	262,144	0	-	-	-	-	MEMLIMIT
	Sequential	1,307,650	1,045,506	3,134,476	0.495823	-	-	TIMEOUT
512	Binary	287,730	25,586	9,262,424	1.613264	3.831807	**5.445071**	SAT
	Commander	784,502	522,358	3,652,158	0.872565	5.799838	6.672404	SAT
	2-Product	473,760	211,616	2,453,672	0.474873	-	-	TIMEOUT
	Binomial	531,441	0	-	-	-	-	MEMLIMIT
	Sequential	2,652,833	2,121,392	6,361,266	1.062963	-	-	TIMEOUT
729	Binary	571,069	39,628	20,558,002	4.206519	60.621817	64.828336	SAT
	Commander	1,590,685	1,059,244	7,413,230	1.253358	27.193685	**28.447043**	SAT
	2-Product	874,009	342,568	4,839,138	0.814480	-	-	TIMEOUT
	Binomial	1,000,000	0	-	-	-	-	MEMLIMIT
	Sequential	4,994,002	3,994,002	11,978,012	1.961011	-	-	TIMEOUT
1000	Binary	1,055,888	55,888	39,300,904	7.273818	17.388254	24.662072	SAT
	Commander	2,998,384	1,998,384	13,968,946	3.674514	16.422169	**20.096683**	SAT
	2-Product	1,529,020	529,020	8,922,032	1.757805	-	-	TIMEOUT

Tài liệu

[1] M. Bezzel, Schachfreund, Berliner Schachzeitung 3 (1848) 363.

[2] F. Nauck, Briefwechsel mit allen für alle, Illustrirte Zeitung 15 (377) (1850) 182.

[3] C. Letavec, J. Ruggiero, The n-queens problem, INFORMS Transactions on Education 2 (3) (2002) 101–103.

[4] I. Rivin, I. Vardi, P. Zimmermann, The n-queens problem, The American Mathematical Monthly 101 (7) (1994) 629–639.

[5] E. M. Clarke, J. M. Wing, Formal methods: State of the art and future directions, ACM Computing Surveys 28 (4) (1996) 626–643.

[6] F. Barber, M. A. Salido, Introduction to constraint programming, Inteligencia Artificial, Revista Iberoamericana de Inteligencia Artificial 20 (2003) 13–30.

[7] S. A. Cook, The complexity of theorem-proving procedures, in: Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC'71), ACM, 1971, pp. 151–158.

[8] J. Marques-Silva, Practical applications of boolean satisfiability, in: 2008 9th International Workshop on Discrete Event Systems, IEEE, 2008, pp. 74–80.

[9] A. Biere, M. Heule, H. van Maaren, T. Walsh (Eds.), Handbook of Satisfiability, Vol. 185, IOS Press, Amsterdam, 2009.

[10] R. Sosic, J. Gu, Efficient local search with conflict minimization: A polynomial-time algorithm for the n-queens problem, IEEE Transactions on Knowledge and Data Engineering 6 (5) (1994) 661–668.

[11] C. Erbas, M. M. Tanik, Generating solutions to the n-queens problem using 2-circulant matrices, Computers & Mathematics with Applications 24 (11) (1992) 89–97.

[12] K. L. Hoffman, M. Padberg, G. Rinaldi, Traveling salesman problem, Encyclopedia of Operations Research and Management Science 1 (2013) 1573–1578.

[13] S. W. Golomb, L. D. Baumert, Backtrack programming, Journal of the ACM 12 (4) (1965) 516–524.

[14] J. R. Bitner, E. M. Reingold, Backtrack programming techniques, Communications of the ACM 18 (11) (1975) 651–656.

[15] L. Simon, Reasoning with propositional logic: From sat solvers to knowledge compilation, A Guided Tour of Artificial Intelligence Research: Volume II (2020) 115–152.

[16] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, S. Malik, Chaff: Accelerating engineering SAT runtimes, in: Proceedings of the 38th Design Automation Conference (DAC'01), ACM, 2001, pp. 530–535.

[17] G. Audemard, L. Simon, Predicting learnt clauses quality in modern SAT solvers, in: Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI'09), 2009, pp. 399–404.

[18] A. Biere, Lingeling, plingeling, picosat and precosat at sat race 2010, FMV Report Series Technical Report 10/1, Johannes Kepler University, 2010.

[19] C. Sinz, Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming (CP 2005), Vol. 3709 of LNCS, Springer, 2005, pp. 827–831.

[20] W. Klieber, G. Kwon, Efficient CNF encoding for selecting at most one item from a group, in: Proceedings of the International Workshop on Constraints in Formal Verification (CFV'07), 2007, pp. 1–9.

[21] J. Chen, A new SAT encoding of the at-most-one constraint, in: Proceedings of the 10th International Conference on Formal Methods in Computer-Aided Design (FMCAD'10), 2010, pp. 268–275.

[22] A. M. Frisch, P. A. Giannaros, SAT encodings of the at-most-k constraint, in: Proceedings of the 3rd International Workshop on Modelling and Reformulating CSPs, 2005, pp. 45–56.

[23] J.-C. Régin, A filtering algorithm for constraints of difference in CSPs, in: Proceedings of the 12th National Conference on Artificial Intelligence (AAAI'94), AAAI Press, 1994, pp. 362–367.